In-App Notification Feature

This document describes the Sprint 10 in-app notification system: what is stored, who is told about what, how a notification reaches an open dashboard in real time over Server-Sent Events, and how the client shows it. All endpoints are under `/api/v1`. Every response uses the standard envelope: `{ "success": true, "message": "...", "data": ... }` on success and `{ "success": false, "message": "...", "error": { "code": "...", "details": ... } }` on failure. The one exception is the live stream, which answers with Server-Sent Events instead of JSON once it is open.

1. Overview

Sprint 10 gave every role a notification inbox that updates while the dashboard is open:

- **28 event types across all six roles** — applications, visits, appointments, government ID reviews, sign-ups waiting for approval, transfers, capacity alerts, tasks, events, donations and admin sign-ups. Each one names who it is for and links to the page where they act on it (Section 3).
- **Real time over Server-Sent Events** — each signed-in dashboard holds one stream open (`GET /notifications/stream`). A new notification is pushed the moment its row is stored, and the bell's badge goes up without a reload (Section 4).
- **The database is the record, the stream only nudges** — every notification is a row first. A tab that was closed or disconnected simply refetches when it reconnects, so nothing is lost if a push is missed.
- **A bell and a Notifications page on every dashboard** — the navbar bell shows the unread count and the latest 10; View all opens `/<role>/notifications` with All/Unread, pages and per-row read/unread (Section 6).
- **Notifications never break the action they report** — `notify()` runs after the action's own write has committed and never throws. A failure is logged; the request still succeeds.
- **An open dashboard stays signed in** — the stream ends when the 15-minute access token expires, and the client renews the token with the refresh cookie and reconnects. Before Sprint 10 the first request after 15 minutes was refused and the user was logged out.
- **An optional chime** — a short sound for each live notification, played by one tab only, with a mute toggle in the bell.

> ***Scope decision — what is in v1:*** *a curated list of events that someone needs to act on or would want to know about. Not notified: favourites, AI matches, health records, preferences and digests. Email (doc 16) stays the channel for account and security messages; a few events — application decisions and government ID reviews — send both an email and an in-app notification.*

The table below lists every endpoint in this document, and the walkthrough or reference entry that covers it.

Endpoint	Auth	Purpose
`GET /notifications`	Bearer (any role)	The caller's notifications, newest first; All or Unread, paged (7.1)
`GET /notifications/unread-count`	Bearer (any role)	The caller's unread total (7.2)
`PATCH /notifications/:id`	Bearer (any role)	Mark one read or unread (7.3)
`PATCH /notifications`	Bearer (any role)	Mark all read (7.4)
`GET /notifications/stream`	Bearer (any role)	The live Server-Sent Events stream (Section 4, 7.5)
`POST /shelters/:id/capacity-alerts`	Bearer (Admin)	Notify a shelter's manager that it is near capacity (5.5, 7.6)

> ***Plain authenticate, not allowPending:*** *a Pending staff member, vet or volunteer is still onboarding and has no dashboard — and so no bell — yet. Every query is scoped to the caller's own userID, so another user's notification answers exactly like one that doesn't exist (404).*

2. Architecture (services/notifications/)

Everything that sends or reads a notification goes through one layer, `services/notifications/`. Services call it after their own database write has committed; the controller for `/notifications` calls it for the inbox and the stream.

2.1 Structure

`services/notifications/index.js` is the **only** import for notifications — the same rule as `services/email/` and `services/ai/`. Nothing else requires `messages.js`, `hub.js`, `inbox.js` or `stream.js` directly (the seed is the one exception: it builds its rows with `messages.js` so they read exactly like real ones).

File	Responsibility
`services/notifications/index.js`	The only import. `notify`, `notifyAfterLookup`, `messages`, the recipient lookups, and re-exports of the inbox and `openStream`
`services/notifications/messages.js`	One builder per `NotificationType`, each returning `{ type, title, body, link }`
`services/notifications/hub.js`	The in-process stream registry: `subscribe`, `publish`, `frame`
`services/notifications/inbox.js`	The REST side: list, unread count, mark one, mark all; `toItem` (the row minus `userID`)
`services/notifications/stream.js`	`openStream`: headers, the opening count, heartbeat, the token-expiry end, cleanup
`config/notifications.js`	Retention, page sizes and the heartbeat interval (Section 8.2)
`controllers/notifications/notifications.controller.js`	Validates query and body, shapes the envelope (incl. the top-level `unreadCount`)
`routes/notifications/notifications.routes.js`	The five `/notifications` routes, mounted at `/api/v1` in `app.js`

2.2 notify(): store, then push

A service reports an event with one call, after its write has committed:

```
await notify([adopterID], messages.visitConfirmed({ shelterName, petName, visitID }));
```

`notify(userIDs, message)` does three things:

1. **Cleans the recipient list** — nulls, non-integers and repeats are dropped (an unassigned vet, a manager who is also on the staff list). With no one left it returns `[]` and writes nothing.
2. **Stores one row per recipient** in a single `createManyAndReturn`. `title`, `body` and `link` are clipped to their column lengths (120, 300, 200) with an ellipsis, so a long pet plus shelter name can never fail the insert.
3. **Pushes each stored row** to its owner's open streams with `hub.publish(userID, "notification", toItem(row))`.

It is awaited — one quick insert, so the rows exist by the time the response goes out — but it **never throws**. The action it reports has already happened, so a failed insert is logged and `[]` is returned; a failed push is logged and the stored rows are still returned. This is the same rule as `queueEmail` in doc 16.

2.3 notifyAfterLookup(): when recipients need a query first

Most hooks need a lookup before they know who to tell or what to say (the shelter's staff, the adopter's name). `notifyAfterLookup(context, prepare)` runs `prepare()`, which resolves to `[userIDs, message]`, and passes the result to `notify`. A failed lookup is logged with `context` (for example "the new visit request 12") and swallowed, for the same reason.

```
await notifyAfterLookup(`the new visit request ${visit.visitID}`, async () => [
  await shelterStaffIDs(shelterID),
  messages.newVisitRequest({ adopterName, petName }),
]);
```

2.4 Recipient lookups

Each lookup returns user IDs to pass straight to `notify`, and only **Active** accounts — a Pending, Deactivated or Closed account has no dashboard to see them on.

Lookup	Returns
`shelterStaffIDs(shelterID)`	Every Active staff member at the shelter, the manager included
`shelterManagerID(shelterID)`	The shelter's manager if Active, else null
`activeAdminIDs()`	Every Active admin
`acceptedAdopterForPet(petID)`	The adopter holding the pet's Accepted application, or null (at most one — a partial unique index)
`signupApproverIDs(kind, shelterID)`	Who approves a sign-up: Volunteer → the shelter's staff; Staff or Veterinarian → the shelter's manager; Manager → the admins

> ***Nobody is told about their own action:*** *a vet who reschedules their own appointment, a manager who reassigns a transfer to themselves, and a staff member who accepts an application are all left out of the recipients.*

2.5 The hub: in-process pub/sub

`hub.js` keeps a `Map` of userID → the set of open stream responses (one per tab). `subscribe(userID, res)` adds one and returns the function that removes it; `publish(userID, event, data)` writes one SSE frame to each of that user's streams. A stream whose write throws has gone away without a `close` yet, and is dropped.

> ***Single instance by design:*** *the hub lives in memory, so it only reaches streams on the same server process — fine for the single Render instance. Scaling out would put Postgres `LISTEN/NOTIFY` behind the same `publish()`. Nothing is lost meanwhile: the rows are stored, and a client refetches when it reconnects.*

Diagram — one notification, from the action to the bell

```
staff confirms a visit (PATCH /visits/:id)
      |
      v
visits.service.js updates the Visit row        (commits)
      |
      v
notify([adopterID], messages.visitConfirmed(...))
      |
      +-- prisma.notification.createManyAndReturn   one row per recipient
      |       +-- insert fails: logged, returns []  (the request still succeeds)
      |
      +-- hub.publish(adopterID, "notification", item)
              |
              +-- adopter has no open dashboard: nothing to write
              |
              +-- each open tab's stream:
                      event: notification
                      data: {"notificationID":41,"type":"VisitConfirmed",...}
                          |
                          v
                useNotificationStream (browser)
                 - unread count + 1  (the bell's badge)
                 - refetch the lists on screen
                 - chime (one tab)
```

3. Event Catalogue

Every notification has a `NotificationType` (PascalCase, an enum in `schema.prisma`) and is built by the matching function in `messages.js`. The text names no dates or times — the server would have to pick a timezone, and the linked page already shows them in the viewer's own. `link` is a client path; where the page can open the item itself it carries a query parameter (Section 6.5).

3.1 Adopter

Type	Sent to	When	Link
ApplicationSubmitted	The adopter	The Stripe webhook created the application after the $15 fee	/adopter/applications
ApplicationApproved	The applicant	Staff/Admin accept the application	/adopter/applications
ApplicationDeclined	The applicant, and every auto-declined applicant	Staff/Admin reject it — or accept another application for the same pet, which declines the pet's other Pending ones — or revoke an approved adoption (revoked wording: "Adoption cancelled")	/adopter/applications
VisitConfirmed	The adopter	Staff/Admin confirm the visit	/adopter/visits?visitID=<id>
VisitCancelled	The adopter	Staff/Admin cancel the visit	/adopter/visits?visitID=<id>
AppointmentScheduled	The adopter with the pet's Accepted application	A vet appointment is created for the pet	/adopter/appointments

Type	Sent to	When	Link
AppointmentChanged	Same	Its date, reason or vet changes (a volunteer swap isn't news)	/adopter/appointments
AppointmentCancelled	Same	Staff/Admin cancel it	/adopter/appointments
GovernmentIdVerified	The ID's owner (any role with one)	A reviewer verifies it	/<role>/profile
GovernmentIdRejected	The ID's owner	A reviewer rejects it — asks for a new upload	/<role>/profile

The two government ID types are listed here but go to whichever role owns the ID; the link uses that role's dashboard.

3.2 Shelter staff

Type	Sent to	When	Link
NewApplication	Every Active staff member at the shelter	An adoption application is paid for	/staff/applications
ApplicationWithdrawn	Same	The adopter withdraws it — or surrenders an approved pet (surrendered wording: "Pet surrendered"; the pet is back as incoming)	/staff/applications
NewVisitRequest	Same	An adopter books a visit	/staff/visits
VisitCancelledByAdopter	Same	The adopter cancels a visit	/staff/visits
SignupReadyForApproval	The approvers (signupApproverIDs)	A Pending staff member, vet or volunteer finishes onboarding — once, not on a repeat call	/staff/volunteers, /staff/team, /staff/vets or /admin/staff
TransferIncoming	The destination's assigned staff member, or its whole team when none	A transfer is initiated — and to the new assignee when the destination manager reassigns it	/staff/transfers
TransferCompleted	The origin's assigned staff member (or team)	The destination accepts the transfer	/staff/transfers
TransferRejected	Same	The destination declines it	/staff/transfers
TransferCancelled	The destination's staff member (or team)	The origin cancels it	/staff/transfers
CapacityAlert	The shelter's manager	An admin clicks Notify Manager on the Overview	/staff/transfers

3.3 Veterinarian, volunteer, donor and admin

Type	Sent to	When	Link
AppointmentAssigned	The vet	An appointment is created for them, or moved to them from another vet	/vet/appointments
AppointmentChanged	The vet who keeps it	Its date or reason changes	/vet/appointments

Type	Sent to	When	Link
AppointmentCancelled	The vet	It's cancelled, or moved to another vet	/vet/appointments
TaskAssigned	Newly added volunteers only	A task is created, or its assignees are edited	/volunteer/tasks
EventAssigned	Newly added volunteers only	An event is created, or its volunteers are edited	/volunteer/events
EventCancelled	Every volunteer on the event	A future event is deleted (tidying away a past one isn't news)	/volunteer/events
AppointmentAssistAssigned	The volunteer	They are added to assist an appointment	/volunteer/appointments
AppointmentAssistCancelled	The volunteer	They are removed, or the appointment is cancelled	/volunteer/appointments
DonationReceived	The donor	The Stripe webhook recorded the donation (receipts aren't emailed)	/donor/history
AdminSignupPending	Every Active admin	A second or later admin registers (the first auto-activates)	/admin/admins

> ***Only the newly added:*** *when staff edit a task's or event's volunteers, the old set is read first and only volunteers who weren't on it are told. If that read fails, the edit still saves and nobody is told — it costs the notification, not the edit. Volunteers removed from a task or event have no type to tell them by in v1.*

4. The Live Stream (Server-Sent Events)

4.1 Why Server-Sent Events

Notifications only ever flow one way, server to browser. Server-Sent Events are exactly that: one long-lived HTTP response that the server keeps writing small text frames into. They need no extra protocol or library on either side, pass through ordinary HTTP infrastructure, and reconnecting is just making the request again. WebSockets would add a two-way channel the feature doesn't use.

4.2 Authentication

The browser's built-in `EventSource` can't send an `Authorization` header, and the access token lives only in memory (never in a cookie). So the client opens the stream with `fetch` and the usual `Authorization: Bearer <accessToken>` header, then reads the response body as a stream (`openNotificationStream` in `logic/api/notificationsApi.ts`, which parses the frames itself). The route uses the same `authenticate` middleware as every other endpoint; a missing or invalid token is a normal JSON 401 before any event is sent.

4.3 Events

Event	Data	When
unread-count	{ unreadCount }	Right after the stream opens, and whenever a mark-read in any of the user's tabs changes the count — so every tab's badge agrees
notification	The item (the row minus userID)	Each new notification for this user

Event	Data	When
token-expired	{}	Just before the server ends the stream at the access token's expiry
: heartbeat	(a comment line, no data)	Every 25 seconds, so proxies and load balancers don't close a quiet connection

Each frame is an `event:` line, one JSON `data:` line and a blank line:

```
event: unread-count
data: {"unreadCount":2}

: heartbeat

event: notification
data: {"notificationID":41,"type":"VisitConfirmed","title":"Visit confirmed",...}
```

4.4 The stream ends at the token's expiry

A token is only checked when a request starts, so a stream could otherwise outlive the token it was opened with — and an account deactivated meanwhile would keep receiving events. `authenticate` now records the token's expiry on the request (`req.tokenExpiresAt`, the JWT's `exp` in milliseconds), and `openStream` sets a timer for it: at expiry it sends `token-expired` and ends the response. The client then renews the token and reconnects (Section 5.4).

4.5 Opening and cleaning up

1. **Headers** — 200 with `Content-Type: text/event-stream`, `Cache-Control: no-cache, no-transform`, `Connection: keep-alive` and `X-Accel-Buffering: no` (stops nginx-style proxies holding events back), flushed at once.
2. **Subscribe before counting** — the response is registered with the hub before the unread count is read, so a notification landing in between still arrives (the badge may count it twice for a moment; the next `unread-count` corrects it).
3. **Opening count** — the `unread-count` event. If the count query fails it is logged and the stream stays open; the client's own list fetch has the count.
4. **Timers** — the heartbeat interval and the expiry timeout start.
5. **Cleanup, exactly once** — on the request's `close` (the client went away), the response's `close` (after `res.end()`) or the response's `error` (a write to a dead socket): both timers stop and the hub entry is removed.

Diagram — one stream's life

```
browser: fetch GET /notifications/stream  (Bearer token)
      |
      v
authenticate  -- no or invalid token --> 401 JSON, no stream
      |  req.tokenExpiresAt = exp * 1000
      v
openStream
 - 200 text/event-stream, headers flushed
 - hub.subscribe(userID, res)
 - event: unread-count
      |
      |   : heartbeat            every 25 s
      |   event: notification    whenever notify() stores a row for them
      |   event: unread-count    after a mark-read in any tab
      |
      +-- the tab closes / logs out / navigates off the dashboard
      |       -> close -> cleanup (timers stopped, unsubscribed)
      |
      +-- the access token expires
              -> event: token-expired, res.end() -> cleanup
              -> the client renews the token and opens a new stream
```

5. Flow Walkthroughs

Each flow names the page, the endpoint, the code that runs and what each person sees.

5.1 A visit request and its confirmation

Flow — adopter books a visit → staff confirm → the adopter is told

1. **Book**: on Visits, the adopter clicks **+ Schedule Visit** and submits `ScheduleVisitPanel`, which calls `POST /visits`. `visits.service.js` `createVisit` stores the visit.
2. **Staff are told**: `notifyAfterLookup` resolves `shelterStaffIDs(shelterID)` and the adopter's name and stores a **NewVisitRequest** for every Active staff member at the shelter. Each staff member with a dashboard open sees the badge go up straight away.
3. **Confirm**: a staff member opens the visit on `/staff/visits` and clicks **Confirm Visit**, which calls `PATCH /visits/:id { visitStatus: "Confirmed" }` → `routes/adopter/visits.routes.js` → `visits.controller.js` → `visits.service.js` `updateVisitStatus`.
4. **The adopter is told**: after the update, `notify([adopterID], messages.visitConfirmed({ shelterName, petName, visitID }))` stores the row and the hub pushes it to the adopter's open tabs. Their badge, the Overview's Notifications tile and the bell's list update without a reload, with a chime.
5. **Open it**: clicking the notification marks it read (`PATCH /notifications/:id { read: true }`) and goes to `/adopter/visits?visitID=12`, where the Visits page opens that visit's details.

Staff can also **cancel** an upcoming visit (Sprint 10 added this — `PATCH /visits/:id { visitStatus: "Cancelled" }` on the same terms as the adopter's own cancel: future and not closed), which sends the adopter **VisitCancelled**. **Completed** sends nothing: the adopter was there.

5.2 Accepting one of several applications

Flow — two applicants, one is accepted

1. **Applied**: each paid application arrives through the Stripe webhook (`finalizeApplication`), which sends the adopter **ApplicationSubmitted** and the shelter's staff **NewApplication**.

2. **Accept**: staff call `PATCH /adoption-applications/:id/status { status: "Accepted" }`. In one `$transaction` the application is accepted, the pet is marked adopted, a `findMany` reads the pet's other Pending applications, and an `updateMany` declines them with an automatic remark.
3. **Told**: after the commit, `notifyApplicationDecision` sends the accepted applicant **ApplicationApproved** and every applicant from that `findMany` **ApplicationDeclined** (the decision emails go out alongside, as in doc 16).
4. **Withdraw: if an adopter withdraws instead, the shelter's staff get ApplicationWithdrawn, with the "Pet surrendered" wording when it was already Accepted (a surrender). A staff revoke of an Accepted application sends the adopter ApplicationDeclined with the "Adoption cancelled" wording. The second wording is a flag on the same builder, so there is still one builder per type.**

5.3 Who hears about an appointment change

`notifyAppointment({ before, after, actorID })` in `staff/appointments.service.js` handles create, edit and cancel for all three people an appointment concerns — the vet, the assisting volunteer and the adopter holding the pet's Accepted application. `before` is null for a new appointment and `after` is null once it is cancelled.

Change	Vet	Volunteer	Adopter
Created	Assigned	Assist assigned	Scheduled
New date or reason	Changed	Changed	Changed
Vet swapped	Old: Cancelled · New: Assigned	—	Changed
Volunteer swapped	—	Old: Assist cancelled · New: Assist assigned	—
Volunteer removed	—	Assist cancelled	—
Cancelled	Cancelled	Assist cancelled	Cancelled
Saved unchanged	—	—	—

Whoever made the change is never told about it — a vet rescheduling their own appointment informs only the volunteer and the adopter.

5.4 Reconnecting and token renewal

Flow — the stream drops, or the token expires

1. **Dropped** (network change, server restart): `useNotificationStream` waits and reconnects, starting at 1 second and doubling up to 30 seconds; the wait resets once a stream opens.
2. **Catch up**: every time a stream opens, the client invalidates all `["notifications"]` queries, so anything stored while it was disconnected appears from the list. The stream is a nudge, not the record.
3. **Token expired**: on `token-expired` (or a 401 when opening), the hook calls `POST /auth/refresh-token` with the refresh cookie, puts the new access token in the auth store (`setToken`) and the effect reconnects with it. The dashboard stays signed in for the refresh token's 7 days.
4. **Two tabs racing**: the refresh token rotates on every use, so a second tab renewing at the same moment can find its cookie already replaced. The renewal is retried once after 1 second, which picks up the new cookie.
5. **Renewal can't help**: a token refused within 10 seconds of a renewal means the account was closed or deactivated meanwhile. The session ends exactly like a 401 from any other request.

5.5 Notify Manager (capacity alert)

1. **See it**: the admin Overview's Capacity Alerts widget lists shelters at 90% utilisation or more, with **Notify Manager** on rows whose shelter has a manager.
2. **Send**: the button calls `POST /shelters/:id/capacity-alerts`. The service checks the shelter exists (404) and has an Active manager (409), then sends that manager a **CapacityAlert** suggesting transfers.
3. **Confirm**: 201 names who was told; the widget toasts "<manager> has been alerted about <shelter>". A 409 (the manager has since been removed or deactivated) shows the server's message.

> ***The one hook that can fail the request:*** *everywhere else a notification is a side effect, but here sending is the whole request — so if the insert stores nothing, the endpoint answers 500 instead of a misleading success.*

6. Client Side

6.1 useNotificationStream

`logic/hooks/useNotificationStream.ts` is mounted once, in the shared `DashboardLayout`, so it runs on every dashboard page of every role and stops when the layout unmounts (logout, or leaving the dashboard). It holds the stream open and folds each event into the TanStack Query cache:

- `unread-count` → sets `["notifications", "unread-count"]` to the server's number.
- `notification` → adds one to that count, plays the chime and invalidates the lists on screen.
- `token-expired` → renews the token and reconnects (Section 5.4).

6.2 Query keys

Key	Holds	Used by
["notifications"]	Everything — one invalidate after a reconnect	useNotificationStream
["notifications", "unread-count"]	A plain number	The bell's badge, the adopter Overview's Notifications tile
["notifications", "list", params]	One page of the list	The bell's dropdown (latest 10), the Notifications page

6.3 The bell

`NotificationBell` (`pages/protected/shared/notifications/`) is handed to `DashboardNavbar` as a slot, so the navbar itself stays content-agnostic. It shows the unread badge (9+ past nine) and, while open, the latest 10 notifications — the list is only fetched while the dropdown is open. Clicking one marks it read and follows its link; **Mark all as read** and **View all** sit in the dropdown, with the sound toggle. It's non-modal like the account menu: an outside click or Escape closes it.

6.4 The Notifications page

`NotificationsPage` is the same page at `/<role>/notifications` on all six dashboards, reached from the bell's View all (not the sidebar menu — the same rule as Profile). All or Unread, 20 per page, newest first; each row can be marked read or unread on its own, and opening one does what the dropdown does. The stream keeps it current while it's on screen.

6.5 Links that open the item

A notification's `link` is followed with the router. Where a page can open the item itself, the link carries a query parameter: the adopter's visit notifications link to `/adopter/visits?visitID=<id>`, and the Visits page opens that visit's details and then clears the parameter. The page watches the parameter instead of reading it once on mount, so it also works when the adopter clicks the notification while already on Visits.

6.6 The chime

`logic/notifications/notificationSound.ts` plays two short soft notes for each live `notification` event — never for the backlog refetched after a reconnect. It is generated with the browser's Web Audio API, so there is no sound file and no library.

- **One tab chimes**: every open tab receives the same notification, so the first to play it records its ID in localStorage and the others stay quiet.
- **Autoplay rule**: browsers keep a page silent until the person has clicked or pressed a key on it, so a chime due before that is skipped.
- **On/off**: the speaker button in the bell's dropdown, remembered per browser (localStorage), on by default.
- **Devices**: the same tones everywhere; volume follows the device. iOS Safari's silent switch mutes it, and a phone browser in the background pauses the page (and the stream) until it's back.

6.7 Other places

- The adopter Overview's **Notifications** tile shows the unread count from the same query as the bell.
- The staff visit detail panel has a red **Cancel Visit** while the visit is upcoming and open.

7. API Reference

Every notification is returned in the same item shape, on the stream and from the REST endpoints — the stored row without its `userID`:

```
{
  "notificationID": 41,
  "type": "VisitConfirmed",
  "title": "Visit confirmed",
  "body": "PetPals Downtown confirmed your visit to meet Biscuit.",
  "link": "/adopter/visits?visitID=12",
  "readAt": null,
  "createdAt": "2026-10-21T14:02:11.000Z"
}
```

7.1 GET /api/v1/notifications

The caller's notifications, newest first (ties broken by the higher ID). Before reading, it deletes the caller's notifications that were read more than 90 days ago — a lazy cleanup with no scheduled job; unread ones are never deleted.

`unreadCount` is the caller's total unread, whatever the filter or page, and sits at the top level beside `pagination` (an extension of the standard envelope) so the bell gets its list and badge from one request.

Request

Field	Value
Method	GET
Auth	Bearer, any role

Field	Value
Query	`unread` — `true` (unread only) or `false`/omitted (all); `page` — integer ≥ 1, default 1; `limit` — 1–50, default 20

Example request

```
GET /api/v1/notifications?unread=true&limit=10
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Notifications retrieved successfully",
  "data": [ { "notificationID": 41, "type": "VisitConfirmed", ... } ],
  "pagination": { "page": 1, "limit": 10, "total": 1, "totalPages": 1 },
  "unreadCount": 1
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in (missing or invalid access token)	UNAUTHORIZED	401
unread isn't 'true' or 'false'	BAD_REQUEST	400
page isn't an integer ≥ 1	BAD_REQUEST	400
limit isn't an integer from 1 to 50	BAD_REQUEST	400

7.2 GET /api/v1/notifications/unread-count

The caller's unread total — what the bell shows on load, before the stream's first event.

Request

Field	Value
Method	GET
Auth	Bearer, any role

Example request

```
GET /api/v1/notifications/unread-count
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Unread count retrieved successfully",
  "data": { "unreadCount": 3 }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in	UNAUTHORIZED	401

7.3 PATCH /api/v1/notifications/:id

Marks one of the caller's notifications read or unread. Marking an already-read one read again keeps its first `readAt`. The new unread count is returned and also pushed to the caller's other open tabs as an `unread-count` event.

Request

Field	Value
Method	PATCH
Auth	Bearer, any role
Body	`{ "read": true }` or `{ "read": false }`

Example request

```
PATCH /api/v1/notifications/41
Authorization: Bearer <accessToken>
{ "read": true }
```

Response

```
200 OK
{
  "success": true,
  "message": "Notification marked read",
  "data": {
    "notification": { "notificationID": 41, "readAt": "2026-10-
21T14:05:40.000Z", ... },
    "unreadCount": 0
  }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in	UNAUTHORIZED	401
id isn't a positive integer	BAD_REQUEST	400
read missing or not a boolean	BAD_REQUEST	400
No such notification — or it belongs to someone else	NOT_FOUND	404

7.4 PATCH /api/v1/notifications

Marks every unread notification of the caller's read, and pushes a zero count to their other tabs. Marking everything unread isn't offered.

Request

Field	Value
Method	PATCH
Auth	Bearer, any role
Body	`{ "read": true }`

Example request

```
PATCH /api/v1/notifications
Authorization: Bearer <accessToken>
{ "read": true }
```

Response

```
200 OK
{
  "success": true,
  "message": "All notifications marked read",
  "data": { "updated": 3, "unreadCount": 0 }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in	UNAUTHORIZED	401
read isn't true	BAD_REQUEST	400

7.5 GET /api/v1/notifications/stream

Opens the caller's live Server-Sent Events stream (Section 4). The response stays open until the client leaves or the access token expires; it is not a JSON envelope. Open it with `fetch` and the Bearer header — `EventSource` can't send one.

Request

Field	Value
Method	GET
Auth	Bearer, any role
Response type	`text/event-stream`

Example request

```
GET /api/v1/notifications/stream
Authorization: Bearer <accessToken>
```

Response

```
200 OK  (Content-Type: text/event-stream)
event: unread-count
data: {"unreadCount":1}

: heartbeat

event: notification
data: {"notificationID":42,"type":"VisitCancelled",...}

event: token-expired
data: {}
```

After `token-expired` the server ends the response; the client renews its token and opens a new stream.

Error Cases

Scenario	Error Code	HTTP Status
Not signed in — a JSON envelope, before any event	UNAUTHORIZED	401

7.6 POST /api/v1/shelters/:id/capacity-alerts

The admin Overview's Notify Manager. Sends the shelter's manager a CapacityAlert suggesting transfers. There is no body and no utilisation check — the admin decides — and each call sends a new alert.

Request

Field	Value
Method	POST
Auth	Bearer, Admin only
Body	None

Example request

```
POST /api/v1/shelters/3/capacity-alerts
Authorization: Bearer <accessToken>
```

Response

```
201 Created
{
  "success": true,
  "message": "Capacity alert sent to the shelter's manager",
  "data": { "managerID": 60, "managerName": "Morgan Lee" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Not signed in	UNAUTHORIZED	401
Not an Admin	FORBIDDEN	403
id isn't a positive integer	BAD_REQUEST	400
No shelter with that ID	NOT_FOUND	404

Scenario	Error Code	HTTP Status
The shelter has no manager, or its manager isn't Active — "<shelter> has no active manager to alert"	CONFLICT	409
The alert couldn't be stored — "please try again"	INTERNAL_SERVER_ERROR	500

7.7 Changes to existing endpoints

Endpoint	Sprint 10 change
`PATCH /visits/:id`	Staff/Admin may now set `Cancelled` too, on the adopter's terms (future, not closed — 409 otherwise). Staff only at their own shelter (403). Confirm and Cancel notify the adopter; the adopter's own Cancel notifies the shelter's staff.
`POST /visits`	Notifies the shelter's staff (NewVisitRequest).
`PATCH /adoption-applications/:id/status`	Accept/Reject/Revoke notify the applicant (Accept also every auto-declined applicant); Withdraw and Surrender notify the shelter's staff.
`POST /webhooks/stripe`	A paid application notifies the adopter and the shelter's staff; a donation thanks the donor.
`PATCH /government-ids/:id/status`	Notifies the ID's owner (Verified or Rejected).
`POST, PATCH /appointments, PATCH /appointments/:id/cancel`	Notify the vet, volunteer and adopter as in Section 5.3.
`POST, PATCH /tasks · POST, PUT, DELETE /events`	Notify newly assigned volunteers; deleting a future event notifies its volunteers.
`POST /transfers, PATCH /transfers/:id/status, PATCH /transfers/:id`	Notify the other side of the transfer, or the new assignee.
`PATCH /staff/me, /vets/me, /volunteers/me /onboarding-complete`	A Pending sign-up finishing onboarding notifies its approvers, once.
`POST /auth/register`	A second or later admin sign-up notifies the active admins.
Every Bearer route (authenticate)	Sets `req.tokenExpiresAt` (the token's exp in ms) for the stream. No change to responses.

8. Data Model and Configuration

8.1 The Notification table

Column	Type	Notes
notificationID	Int, PK	Auto-increment
userID	Int, FK → Users	`ON DELETE CASCADE` — a deleted account's notifications go with it
type	NotificationType	One of the 28 values in Section 3
title	VARCHAR(120)	Clipped by notify() if longer
body	VARCHAR(300)	Clipped by notify() if longer
link	VARCHAR(200), null	A client path

Column	Type	Notes
readAt	Timestamp, null	null = unread
createdAt	Timestamp	Defaults to now

Indexes `(userID, createdAt)` for the newest-first list and `(userID, readAt)` for unread counts. Migration `20261018120000_in_app_notifications` **replaced** the original Notification table from the first database design — scheduled reminders with no foreign key, never used — and dropped its three enums. It was written by hand, following the project's rule of never running `prisma migrate dev` (PostGIS drift).

8.2 Settings

Setting	Value	Where	Purpose
READ_RETENTION_MS	90 days	config/notifications.js	Read notifications older than this are deleted when their owner next opens the list
DEFAULT_PAGE_LIMIT / MAX_PAGE_LIMIT	20 / 50	config/notifications.js	GET /notifications page size
STREAM_HEARTBEAT_MS	25 s	config/notifications.js	Keeps quiet connections open through proxies
Column limits	120 / 300 / 200	services/notifications/index.js	Clipping for title, body and link
RETRY_MIN_MS → RETRY_MAX_MS	1 s → 30 s	useNotificationStream.ts	Reconnect backoff
REFRESH_RETRY_DELAY_MS	1 s	useNotificationStream.ts	The one renewal retry (two tabs racing)
RENEWAL_GUARD_MS	10 s	useNotificationStream.ts	A 401 this soon after renewing ends the session
DROPDOWN_LIMIT / PAGE_SIZE	10 / 20	NotificationBell.tsx / NotificationsPage.tsx	Rows in the bell and per page
transactionOptions	10 s wait / 15 s	config/prisma.js	Every Prisma transaction's limits (8.3)

8.3 Prisma transaction limits

Sprint 10's integration journeys showed that Prisma's default transaction limits — 2 seconds to get a connection, 5 seconds to finish — were too tight over the remote Supabase link, where opening a fresh connection alone can take a few seconds. A sign-up or a transfer would then fail with a 500. The Prisma client in `config/prisma.js` now sets 10 seconds and 15 seconds for every transaction, including the batch `$transaction([...])` form, which the PostgreSQL driver adapter also runs as a real transaction. When the database is quick these limits never come into play.

8.4 Seed data

The seed (which wipes the database first) stores notifications covering every type, built with `messages.js` so they read exactly like real ones and spread across the main logins so every dashboard's bell has something, read and unread mixed. The adopter's visit notices point at Emelie's real seeded visits (Biscuit, Confirmed; Cleo, Cancelled), so their links open a visit that exists. Nothing new is needed in `.env` — notifications have no external provider.

9. Testing

9.1 Unit tests

Prisma and the hub are mocked; the message text is the real one. `server/src/tests/unit/notifications/`:

Suite	Covers
messages.test.js	Exactly one builder per NotificationType (read from schema.prisma); every row fits its columns, has no blank text and links to a client path; role-dependent links, no-pet wording, task names, dollar amounts, the visit links
notify.test.js	Recipient clean-up, clipping, the push per row, failed insert and failed push are logged and swallowed, notifyAfterLookup, every recipient lookup
hub.test.js	Events reach every tab of the right user and no one else; a broken stream is dropped
notifications.endpoints.test.js	Own-only 404, the unread filter and total, paging, retention, mark read/unread/all, the pushes to other tabs, every 400, a database error → 500
stream.test.js	Headers, the opening count, heartbeat, token-expired and the end at expiry, cleanup exactly once on every kind of close
hooks.test.js	finalizeApplication, createVisit, finalizeDonation, the admin sign-up, and every notifyAppointment case

The other hooks are tested inside their service's existing suite, in a Notifications block (applications, visits, tasks, events, transfers, government IDs, the three onboarding suites, shelters). They share `notificationsMock.js` as their `jest.mock` target: `notify` and the lookups are mocks to assert on, `notifyAfterLookup` behaves like the real one, and `messages` is the real module. Together the notification service and controller code is 100% covered.

9.2 Integration journeys

`server/src/tests/integration/notifications/sprint10Journeys.test.js` drives each action as one user against the live test database and reads the other user's inbox. Applications arrive through a locally signed Stripe webhook, and everything created is removed afterwards (notifications cascade with their user).

- **Two applications, one accepted** — staff hear of both, the winner is approved, the runner-up declined, the deciding staff member gets nothing.
- **Visit** — the request reaches both staff members; the confirmation reaches the adopter, who reads it; another user's notification is a 404; one staff member's mark-all touches only their own inbox.
- **Transfer** — the destination's manager hears it's coming; completing it tells the initiator.
- **Live stream** — over a real server (`app.listen(0)`) and `fetch`: the stream opens with the count, the confirmation arrives the moment staff confirm, and a mark-read elsewhere pushes the new count. Without a token it's a 401.

The journeys take two to three minutes over the remote database, so the file allows 180 seconds per test.

9.3 Manual check

The part automated tests can't show was checked in two browsers on 21 October 2026: a staff action updates the adopter's bell within about a second without a reload; notifications missed while away from the dashboard, or while the server was restarted, appear on return; logging out closes the stream; the visit notification opens that visit; and the chime plays.